CO3 ASSESSMENT TOOL 1

CODE CHALLENGE

NAME: N. PARNITA

REG.NO: 192525322

COURSE: Design and Analysis of Algorithms – CSA0617

Q1. MINIMUM COST CLIMBING STAIRS

Problem Statement

A staircase has a cost associated with each step. A person can climb either one or two steps at a time. The person can start from either the first or second step. The objective is to reach the top with minimum total cost. Dynamic Programming is used to compute the minimum cost path.

Python Program

```
def min_cost_climbing_stairs(cost):
    n = len(cost)

    dp = [0] * (n + 1)

    dp[0] = 0
    dp[1] = 0

    for i in range(2, n + 1):
        dp[i] = min(dp[i - 1] + cost[i - 1],
                    dp[i - 2] + cost[i - 2])

    return dp[n]


n = int(input("Enter number of steps: "))
```

```
cost = list(map(int, input("Enter cost values: ").split()))

print("Min Cost =", min_cost_climbing_stairs(cost))
```

Sample Input

```
Enter number of steps: 3
Enter cost values: 10 15 20
```

Sample Output

```
Min Cost = 15
```

Analysis

- Dynamic Programming stores the minimum cost required to reach each step.
- At each step, the person can arrive from either the previous step or two steps before.
- The minimum of these two possible costs is selected.
- **Time Complexity:** O(n)
- **Space Complexity:** O(n)

Conclusion

Dynamic Programming efficiently finds the minimum cost required to reach the top by storing previously calculated minimum costs and avoiding repeated calculations.

main.py

```python
def min_cost_climbing_stairs(cost):
    n = len(cost)

    dp = [0] * (n + 1)

    dp[0] = 0
    dp[1] = 0

    for i in range(2, n + 1):
        dp[i] = min(dp[i - 1] + cost[i - 1],
                    dp[i - 2] + cost[i - 2])

    return dp[n]


n = int(input("Enter number of steps: "))
cost = list(map(int, input("Enter cost values: ").split()))

print("Min Cost =", min_cost_climbing_stairs(cost))
```

```
Enter number of steps: 3
Enter cost values: 10 15 20
Min Cost = 15
```

Q2. HOUSE ROBBER PROBLEM

1. Problem Statement

A robber wants to steal money from houses arranged in a line. However, the robber cannot rob two adjacent houses because it triggers an alarm. The objective is to **maximize the total amount of money stolen** using Dynamic Programming. The problem statement and sample are given in the reference document.

2. Dynamic Programming Approach

For each house, there are two choices:

- **Rob the current house:** Add its money to the maximum amount obtained from two houses before.
- **Skip the current house:** Keep the maximum amount obtained from the previous house.

Therefore:

```
dp[i] = max(dp[i-1], dp[i-2] + arr[i])
```

3. Python Program

```
def house_robber(arr):
    n = len(arr)

    if n == 0:
        return 0
    if n == 1:
        return arr[0]

    dp = [0] * n

    dp[0] = arr[0]
    dp[1] = max(arr[0], arr[1])

    for i in range(2, n):
        dp[i] = max(dp[i - 1], dp[i - 2] + arr[i])

    return dp[n - 1]


n = int(input("Enter number of houses: "))
arr = list(map(int, input("Enter money values: ").split()))

print("Max Loot =", house_robber(arr))
```

4. Sample Input

```
Enter number of houses: 4
Enter money values: 2 7 9 3
```

5. Sample Output

```
Max Loot = 11
```

The maximum loot is **11**, obtained by robbing houses containing **2 and 9**.

6. Complexity Analysis

- **Time Complexity:** O(n)
- **Space Complexity:** O(n)

7. Conclusion

Dynamic Programming finds the maximum possible loot by storing the best result for each house and ensuring that two adjacent houses are never selected.

main.py

```python
def house_robber(arr):
    n = len(arr)

    if n == 0:
        return 0
    if n == 1:
        return arr[0]

    dp = [0] * n

    dp[0] = arr[0]
    dp[1] = max(arr[0], arr[1])

    for i in range(2, n):
        dp[i] = max(dp[i - 1], dp[i - 2] + arr[i])

    return dp[n - 1]


n = int(input("Enter number of houses: "))
arr = list(map(int, input("Enter money values: ").split()))

print("Max Loot =", house_robber(arr))
```

```
Enter number of houses: 4
Enter money values: 2 7 9 3
Max Loot = 11
```

Q3. MAXIMUM PRODUCT SUBARRAY

1. Problem Statement

Given an array of integers, find the contiguous subarray that has the **maximum product**. The array may contain negative numbers, which can change the product value. Dynamic Programming is used to efficiently determine the maximum possible product.

2. Dynamic Programming Approach

For each element, maintain two values:

- `max_product` – maximum product ending at the current position.
- `min_product` – minimum product ending at the current position.

The minimum product is also required because multiplying a negative number by the minimum negative product can produce the maximum positive product.

3. Python Program

```
def max_product_subarray(arr):
    max_product = arr[0]
    min_product = arr[0]
    result = arr[0]

    for i in range(1, len(arr)):
        if arr[i] < 0:
            max_product, min_product = min_product, max_product

        max_product = max(arr[i], max_product * arr[i])
        min_product = min(arr[i], min_product * arr[i])

        result = max(result, max_product)

    return result


n = int(input("Enter number of elements: "))
```

```
arr = list(map(int, input("Enter array elements: ").split()))

print("Max Product =", max_product_subarray(arr))
```

4. Sample Input

```
Enter number of elements: 5
Enter array elements: 2 3 -2 4 -1
```

5. Sample Output

```
Max Product = 48
```

The maximum product is **48**, obtained from the subarray:

```
[2, 3, -2, 4, -1]
```

6. Complexity Analysis

- **Time Complexity:** O(n)
- **Space Complexity:** O(1)

7. Conclusion

Dynamic Programming efficiently finds the maximum product by maintaining both the maximum and minimum products ending at each position. This allows negative values to be handled correctly. The reference document specifies the sample input and expected output as `Max Product = 48`.

Run | Debug | Stop | Share | Save | { } Beautify

main.py

```python
def max_product_subarray(arr):
    max_product = arr[0]
    min_product = arr[0]
    result = arr[0]

    for i in range(1, len(arr)):
        if arr[i] < 0:
            max_product, min_product = min_product, max_product

        max_product = max(arr[i], max_product * arr[i])
        min_product = min(arr[i], min_product * arr[i])

        result = max(result, max_product)

    return result


n = int(input("Enter number of elements: "))
arr = list(map(int, input("Enter array elements: ").split()))

print("Max Product =", max_product_subarray(arr))
```

```
Enter number of elements: 4
Enter array elements: 5 -2 4 2
Max Product = 8
```